

Threads, Race Conditions, and Integer-based Vulnerabilities

CS 465 · Introduction to Security and Privacy · Fall 2026

BYU



Why Software Security Matters to You

Most of you will be software developers, and this is where security touches you

Vulnerabilities you introduce affect customers

Cheaper to fix while writing code than after a security incident

Goal: learn to think like an attacker

You cannot defend code you have never tried to break.

Software Security Is Not Security Software

Software security

Building software that behaves properly when someone is attacking it

a parser, a web handler, a firmware updater

Security software

Products whose job is security.

firewalls, antivirus, crypto libraries, password managers

You can deploy every security product on the market and still ship an exploitable program

WHY WE KEEP STUDYING C

C is where these vulnerabilities are easiest to see, the deployed code full of them is still running, and the same patterns resurface in other languages.

AI Cuts Both Ways

New ways to get it wrong

Generated code repeats insecure patterns at scale and carries flaws out of its training data.

Suggestions look confident whether or not they are safe.

Agentic tools add prompt injection and over-broad permissions.

New ways to get it right

Useful for code review, fuzzing harnesses, and triaging static-analysis output.

Good at the tedious pass humans skip: verifying arithmetic, bounds checking

They can propose the patch — which you still have to read.

Either way, the responsibility is yours

You sign off on the code that ships

When Is a Bug a Vulnerability?

An ordinary bug

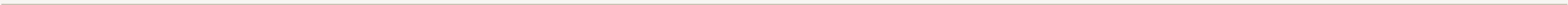
The program does the wrong thing.
Someone files a ticket, you fix it in the next release.

A vulnerability

The wrong thing gives an attacker a foothold — code execution, data, or a way to build a full exploit.

Threads

Lightweight way to run multiple tasks concurrently within a single process



What Threads Share, and What They Don't

ONE PROCESS

private to each thread

**Thread 1
stack
registers, PC**

**Thread 2
stack
registers, PC**

**Thread 3
stack
registers, PC**

code (text segment)

shared by every thread

Heap · malloc'd memory

Global and static variables

Open files and sockets

Shared resources are where race conditions can occur

Creating a Thread

```
pthread_t thread;  
  
pthread_create(&thread,    // 1  
              NULL,       // 2  
              &doit,      // 3  
              &p);        // 4
```

The four arguments

- 1 pointer to your pthread_t handle
- 2 attributes; NULL for defaults
- 3 the function the thread runs
- 4 the argument passed to it

The scheduler decides who runs, and in what order

All threads are runnable at once. Run the program twice and you may get two different answers.

[pthread_create\(\) man page](#)

Waiting for a Thread to Finish

```
pthread_join(thread,    // 1  
                NULL);  // 2
```

The two arguments

- 1 the thread you want to wait for
- 2 where to put its return value, or NULL if you do not want it

Join is how the parent waits

The call blocks until that thread finishes. Skip it and main can return first, tearing down the process while your threads are still working.

[pthread_join\(\) man page](#)

Which Thread Am I?

```
cout << pthread_self();
```

Why you want it

Returns the calling thread's own id.

[pthread_self\(\) man page](#)

Threads Example

- See `threads-example` in race directory of [software-security](#) repo

Race Conditions

Program behavior depends on unpredictable timing of concurrent events

Race Condition Example

- See `inventory` in race directory of [software-security](#) repo

inventory • Where the Item Goes Missing

THREAD A	THREAD B	inventory
reads inventory → 1		1
	reads inventory → 1	1
1 > 0, so sell		1
	1 > 0, so sell	1
writes 0		0
	writes 0	0

Two items sold. One item decremented. The inventory is corrupted.

To Prevent a Race Condition

- Make the program run deterministically by locking critical sections of code
- Use a mutex or semaphore

Mutexes and Critical Sections

```
pthread_mutex_t mutex;  
  
pthread_mutex_init(&mutex, NULL);  
  
pthread_mutex_lock(&mutex);  
  
    // critical section  
  
pthread_mutex_unlock(&mutex);
```

A lock on shared memory

One thread holds the mutex at a time. Every other thread waits at the lock call until it is released.

The critical section

The region between lock and unlock. Covers smallest portion of code that must be executed by one thread at a time

Every thread touching that data must use the same mutex

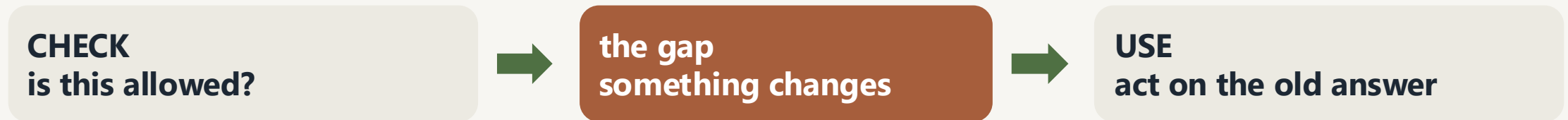
man pages: [pthread_mutex_init\(\)](#) [pthread_mutex_lock\(\)](#)
attributes: [Stack Overflow on mutex attributes](#)

Mutex Example

- See `inventory-fixed` in race directory of [software-security](#) repo
- See `inventory-monitor` for a monitor-like pattern

TOCTOU

- Time of check, time of use
- You check a condition, then act on it — and the world changes in between



No threads required. The two halves can be in one program, in two processes, or in a program and an attacker running alongside it — all that matters is that someone else can act inside the gap.

The access() / open() Race

THE SETUID PROGRAM · runs as root

```
access("/tmp/f", W_OK)  
may the real user write this? → yes
```

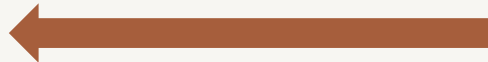


```
open("/tmp/f", O_WRONLY)  
opens /etc/passwd, as root
```

THE ATTACKER · ordinary user

```
rm /tmp/f; ln -s /etc/passwd /tmp/f  
the name now points somewhere else
```

attacker acts inside the gap



Time of check differs from time of use

A Pathname Is Not a Resource

Resolving a name to an object happens every time — and the answer can change between calls.

Symbolic links

A name that points at another name. Resolved fresh on every call, so it can be changed between your check and your use.

Hard links

A second name for the same inode. Permissions follow the inode, not the name you used.

Temp files

A world-writable directory means an attacker can create, delete, and replace names before you get there.

WHAT ACTUALLY HELPS

Open the resource once, then work from the file descriptor — it names the object, not the path.

Prefer calls that check and act in one step: `O_CREAT | O_EXCL`, `mkstemp()`, `openat()`.

General Race vs Data Race

General race condition

Any situation where the outcome depends on timing you do not control. The TOCTOU file race is one — two processes, no shared memory, no threads.

Data race

Two threads touch the same memory, at least one of them writing, with no synchronization between them.

DATA RACE

```
if (inventory > 0)    // read
    inventory = inventory - 1; // modify,
write
```

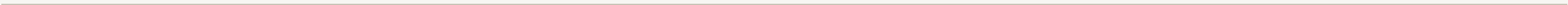
Three steps, not one. Any thread can be interrupted between them — and then two threads both read the same value, both decide there is stock, and both sell it.

Current CVEs

- [Search "toctou" on cve.org](#)
- [Search "underflow" on cve.org](#)

Integer-based Vulnerabilities

Errors in integer values, ranges, arithmetic, or conversions cause a program to behave unsafely



Integers: Fixed Width, Fixed Range

TYPE	BITS	SIGNED RANGE	UNSIGNED RANGE
<code>int8_t</code> / <code>uint8_t</code>	8	-128 ... 127	0 ... 255
<code>int16_t</code> / <code>uint16_t</code>	16	-32,768 ... 32,767	0 ... 65,535
<code>int32_t</code> / <code>uint32_t</code>	32	-2.15e9 ... 2.15e9	0 ... 4.29e9
<code>int64_t</code> / <code>uint64_t</code>	64	-9.2e18 ... 9.2e18	0 ... 1.8e19

WHY THIS MATTERS

Every C integer has a fixed width. Arithmetic that exceeds the range produces a different number than expected, quietly, with no exception and no warning.

Signed Integers and Sign Extension

THE SAME BITS, TWO READINGS

8 BITS	AS int8_t	AS uint8_t
0000 0001	1	1
0111 1111	127	127
1000 0000	-128	128
1111 1111	-1	255

The top bit carries the sign. Nothing in the bits says which reading is correct — the type does.

WIDENING A VALUE

```
int8_t x = -1;
int16_t y = x; // sign extension
sign bit is copied into the new bits — still -1
```

```
uint8_t a = 255;
uint16_t b = a; // zero extension
zeros fill the new bits — still 255
```

Which extension happens depends on the source type — get it wrong and a small negative becomes a huge positive.

When Signed Meets Unsigned

```
int len  = -1;
unsigned int size = 64;

if (len < size)          // guess what happens
    memcpy(dst, src, len);
```

len converts to unsigned: -1 becomes 4,294,967,295, the comparison is false, and memcpy gets a huge length.

Integer promotion

Anything narrower than int is widened to int before the arithmetic even starts.

Usual conversions

Mix signed and unsigned of the same width, and the signed value converts.

A negative length is a classic door for an attacker

Attacker-supplied sizes, counts, and offsets are a common source of vulnerabilities. A bounds check written with the wrong signedness can be an open door.

What C Actually Promises

Unsigned wraps

```
0xFFFF + 1 → 0
```

Defined behavior: arithmetic is modulo 2^n .
The program keeps running with a number nobody expected — no signal, no error.

Signed overflows

```
INT_MAX + 1 → ???
```

Undefined behavior. The compiler may assume it never happens and optimize your safety check away entirely.

Nothing is checked for you

C performs the arithmetic and hands you whatever the bits say. If you want to know that a sum, product, or difference stayed in range, you have to ask before you compute — not after.

Integer Overflow Example

- See `rectangular` in integer directory of [software-security](#) repo

rectangular · Overflow on Multiplication

```
uint16_t rows, cols;    // from the user

buf = malloc(rows * cols);

for (i = 0; i < rows * cols; i++)
    buf[i] = read_byte();
```

FEED IT LARGE VALUES

rows	1,024
cols	4,096
true product	4,194,304
what fits in 16 bits	0
malloc receives	0 bytes

A tiny buffer and a very long loop

The multiplication wrapped, so the allocation is far smaller than you thought it would be. The program ends up writing in uninitialized memory.

Integer Underflow Example

- See `waiting-list` in integer directory of [software-security](#) repo

waiting-list • Predict the Underflow

```
unsigned int waiting = 0;

void leave_queue(void) {
    waiting = waiting - 1;
}

// how many people are waiting now?
```

Predict first

Nobody is in the queue, and one person leaves.

What does the program report now?
What if waiting were signed?

```
0 - 1 → 4,294,967,295    // 32-bit unsigned wraps to its maximum
```

Now imagine that value used as a loop bound, an array index, or an allocation size.

Five Ways Integers Go Wrong

CLASS	WHAT HAPPENS	EXAMPLE
Overflow	value climbs past the maximum and wraps	<code>0xFFFF + 1 → 0</code>
Underflow	value drops below the minimum and wraps	<code>0u - 1 → 4294967295</code>
Signedness mismatch	signed and unsigned compared or assigned	<code>-1 < 64u</code> is false
Narrowing loss	assigned to a smaller type, high bits dropped	<code>(uint8_t)300 → 44</code>
Sign-extension error	negative value widened, then read as unsigned	<code>(unsigned)(int8_t)-1 → huge</code>

Worth memorizing — in the synthesis exercise you will be naming these.

Integer Bugs Bite Indirectly

The wrong number is never the damage. It is what the program does next with that number.

Under-allocation

The size computation wraps, malloc returns a buffer far too small, and the writes that follow run off the end.

Out-of-bounds index

A wrapped or sign-flipped value is used as an array index and lands outside the object.

Enormous allocation

An integer underflow can cause the program to ask for gigabytes — denial of service, or a NULL nobody checks.

Runaway loops

A bound that wrapped turns a short loop into a very large one, writing far past where it should stop.

Every one of these ends up as a memory-safety bug — which is why integer errors show up in exploit write-ups far more often than their share of the code would suggest.

Current CVEs

- [Search "overflow" on cve.org](#)
- [Search "underflow" on cve.org](#)

Avoiding Integer Vulnerabilities

Validate first

Check ranges on anything that came from outside before you do arithmetic with it — not after.

Checked math

Safe integer libraries and built-ins that report overflow instead of silently wrapping.

Right-sized types

`size_t` for sizes, explicit `int32_t` / `uint32_t`. Be deliberate about width and signedness.

Tools

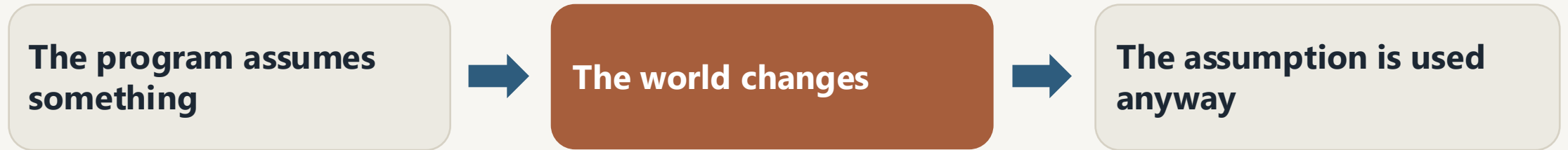
Compiler flags, sanitizers, static analysis — they find what review misses.

A WIDER TYPE IS NOT A FIX

64 bits moves the edge; it does not remove it. If the value still comes from an attacker and is still unchecked, you have bought yourself a larger number to be wrong about.

A language with checked arithmetic would help — but the code we must secure is C.

The Pattern



THE BUG

THE ASSUMPTION

WHAT WAS ACTUALLY TRUE

TOCTOU file race	this path is safe to write	the name points elsewhere
Data race	inventory is still 1	another thread took it
Integer-based vulnerabilities	this sum fits in an int	it wrapped to something small

Name the Bug, Name the Defense

PICK ONE: TOCTOU · DATA RACE · OVERFLOW · UNDERFLOW · CONVERSION ERROR

- | | |
|---|--|
| 1 | A cart quantity is stored unsigned; the code subtracts the returned items without checking. |
| 2 | A privileged tool calls stat() on a path, likes what it sees, then fopen()s the same path. |
| 3 | A signed char holding -3 is passed to a function expecting size_t and used as a length. |
| 4 | A length field of 0xFFFFFFFF arrives; the code adds 1 for the null terminator, then mallocs. |
| 5 | Two worker threads both read a counter at 5, increment it, then both write 6. |

For each one: which class, and what one change would you make?

Name the Bug, Name the Defense

1. **Underflow** — Check that the returned quantity does not exceed the cart quantity before subtracting.
 2. **TOCTOU race** — Open the file safely and perform authorization checks on the resulting file descriptor rather than checking and reopening the pathname.
 3. **Conversion error** — Validate that the length is nonnegative and within range before converting it to `size_t`.
 4. **Overflow** — Check that `length + 1` cannot exceed the maximum representable `size_t` value before adding or allocating.
 5. **Data race** — Protect the read–modify–write operation with a mutex or use an appropriate atomic operation.
-